PROJECT CONTEXT

Version: 1.0

Project Name

Secure Dance Academy Management System

Module

7032CE Secure Design and Development

Objective

Develop a secure, production-quality web application that demonstrates secure software engineering principles while satisfying all assessment requirements for Design, Development, Security Testing and Formal Methods.

This project should resemble software built by a professional engineering team rather than a student coursework submission.

Project Vision

The Secure Dance Academy Management System will become a modern Software as a Service (SaaS) platform for managing dance academies with children and adult performers.

The system should provide secure, reliable and scalable management of artists, coaches, parents, attendance, injuries, performances and administrative activities.

Every engineering decision should balance usability, security, maintainability and future scalability.

Product Goals

Build software that is:

Secure

Reliable

Professional

Maintainable

Accessible

Scalable

Well documented

Fully tested

Easy to extend

Suitable for production

Suitable for academic assessment

Success Criteria

The project succeeds when:

Every functional requirement is implemented.

Security controls protect sensitive information.

Architecture follows modern engineering practices.

Testing demonstrates software quality.

Formal methods verify important workflows.

Documentation explains every major decision.

The project is suitable for deployment.

The project demonstrates distinction-level engineering quality.

Primary Users

Administrator

Coach

Artist

Parent

Every user should receive a dedicated experience with role-based permissions.

Core Features

Authentication

User Management

Role Management

Artist Management

Coach Management

Parent Management

Attendance

Performance Tracking

Injury Records

Medical Information

Emergency Contacts

Reports

Notifications

Audit Logs

Profile Management

Dashboard

Search

Filtering

Pagination

Application Settings

Technology Stack

Frontend

Next.js 15

TypeScript

Tailwind CSS

shadcn/ui

React Hook Form

Zod

Backend

Next.js Route Handlers

Server Actions

Prisma ORM

Database

Supabase PostgreSQL

Alternative Database

MySQL

Authentication

Supabase Authentication

Alternative Authentication

JWT with HTTP-only Secure Cookies

Deployment

Vercel

Docker

Docker Compose

Testing

Jest

Playwright

Postman

OWASP ZAP

SonarQube

Version Control

Git

GitHub

Software Architecture

The application must follow Feature-Based Clean Architecture.

Every feature should remain independent.

Business logic should remain separate from UI.

Database logic should remain separate from business logic.

Security should exist at every layer.

Avoid monolithic code.

Prefer modular design.

Engineering Principles

Secure by Design

Privacy by Design

Clean Architecture

SOLID Principles

DRY

KISS

YAGNI

Separation of Concerns

Least Privilege

Defense in Depth

Fail Secure

Zero Trust

Composition over inheritance

Every engineer should follow these principles.

Security Objectives

Protect user identities.

Protect personal information.

Protect medical information.

Protect attendance records.

Protect performance statistics.

Protect audit logs.

Protect administrative functions.

Reduce attack surface.

Prevent unauthorized access.

Support privacy legislation.

Design Principles

Interfaces should be:

Simple

Consistent

Responsive

Accessible

Fast

Professional

Easy to understand

Easy to navigate

Reduce user effort.

Reduce mistakes.

Support productivity.

Database Principles

Normalize data.

Maintain integrity.

Enforce relationships.

Use constraints.

Use indexes.

Support auditing.

Support future growth.

Never duplicate information unnecessarily.

API Principles

RESTful design.

Consistent naming.

Predictable responses.

Centralized validation.

Role-based authorization.

Meaningful error messages.

Complete documentation.

Every endpoint should have one clear purpose.

Testing Philosophy

Testing exists to increase confidence.

Every important feature should be verified.

Testing includes:

Unit Testing

Integration Testing

API Testing

End-to-End Testing

Security Testing

Regression Testing

Accessibility Testing

Performance Testing

Formal Methods

Critical workflows should be modelled.

Recommended workflows:

Authentication

Attendance

Password Reset

Role Assignment

Medical Record Access

Every model should include:

FSM

Petri Net

Reachability

Deadlock Analysis

Safety

Liveness

Documentation Standards

Every feature requires documentation.

Every API requires documentation.

Every diagram requires explanation.

Every architectural decision requires justification.

Every security decision requires justification.

Every important implementation should be traceable back to a requirement.

Assignment Priorities

Priority 1

Security

Priority 2

Architecture

Priority 3

Correctness

Priority 4

Testing

Priority 5

Documentation

Priority 6

Performance

Priority 7

Visual polish

Never sacrifice higher priorities to improve lower priorities.

Project Deliverables

Complete Web Application

Database Schema

API Documentation

Architecture Documentation

Security Documentation

Testing Evidence

Formal Methods Documentation

Deployment Guide

Developer Guide

User Guide

Presentation Material

Final Report

GitHub Repository

Constraints

Maintain professional coding standards.

Avoid unnecessary complexity.

Never duplicate business logic.

Never expose sensitive information.

Use modern development practices.

Prefer maintainability over shortcuts.

Build features completely before moving forward.

Document every important decision.

Definition of Success

This project should satisfy three independent goals.

It should earn distinction-level marks.

It should demonstrate professional software engineering practices.

It should become a portfolio-quality project suitable for showcasing full stack development, secure software engineering and system architecture skills.

Every engineer, every guide and every task should align their decisions with these objectives.